

QuicknodeEarn Audit



May 8, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
QuicknodeEarn	5
Off-Chain Components	5
Security Model and Trust Assumptions	6
Low Severity	8
L-01 Owner Can Drain User Principal Despite Documented Trust Model	8
L-02 Unconstrained mintRecipient And destinationCaller At CCTP Burn Entry Points	9
L-03 Relay Can Redirect Bridged Deposits To An Arbitrary Beneficiary	11
L-04 relayAndDeposit Strands Excess Minted USDC	12
L-05 Incomplete Input Validation Across Multiple Functions	13
L-06 Vault De-Whitelisting Blocks Operations On Existing User Positions	16
L-07 Executor And Relay Can Deposit On A User's Behalf Without Their Share Approval	17
L-08 Documentation Inconsistent With Implementation	19
L-09 Proxy Variant Does Not Use Upgradeable OpenZeppelin Or ERC-7201 Namespaces	22
L-10 abi.encodeWithSelector Used Instead Of Type-Safe abi.encodeCall	23
L-11 OpenZeppelin Dependency Version Skew Between Hardhat And Foundry	24
Notes & Additional Information	25
N-01 Use ReentrancyGuardTransient For Per-Call Gas Savings	25
N-02 Redundant forceApprove Calls In CCTP Burn Loop	26
N-03 Code Improvements	27
N-04 Missing Documentation On Local External Interfaces	28
Conclusion	30
Appendix	31
Issue Classification	31

Summary

Type	DeFi	Total Issues	15 (14 resolved, 1 partially resolved)
Timeline	From 2026-04-20 To 2026-04-29	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	11 (10 resolved, 1 partially resolved)
		Notes & Additional Information	4 (4 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [quiknode-labs/earn-smart-contracts](#) repository at the [271f2c4](#) commit.

In scope were the following files:

```
contracts/QuicknodeEarn.sol  
contracts/QuicknodeEarnProxy.sol
```

The two contracts share the same business logic. Any business logic issue raised in this report applies equally to both contracts, even where only one is referenced in the body of the issue.

System Overview

QuicknodeEarn is a non-custodial yield optimizer for USDC on EVM chains. It routes user capital between whitelisted Morpho ERC-4626 vaults and the Aave V3 Pool to track the highest supply APY on a given chain, and integrates Circle's Cross-Chain Transfer Protocol v2 (CCTP v2) for moving USDC between chains. Users grant ERC-20 approvals to the contract but do not transfer principal to it outside of a single atomic transaction. Vault shares and aTokens are always held directly in the user's wallet.

QuicknodeEarn

The **QuicknodeEarn** contract is the sole on-chain entry point for user interactions. Two variants are present in the repository: a non-proxy **QuicknodeEarn.sol**, and a UUPS-upgradeable variant **QuicknodeEarnProxy.sol**.

Main responsibilities:

- Whitelist management for Morpho vaults and the Aave V3 Pool.
- Atomic **rebalance** and **withdrawAndBridge** operations initiated by an off-chain rebalancer.
- Relay and deposit of CCTP messages arriving from other chains via **relayAndDeposit**.
- Self-service strategy creation and exit through **selfBatchDeposit** and **selfBatchWithdraw**, including optional CCTP burns for cross-chain movement.
- Performance fee collection in the form of vault shares, transferred from the user to the contract before being swept to the owner.

Off-Chain Components

The contract is designed around three off-chain systems:

- A **rebalancer** that monitors yield across chains and vaults, computes optimal allocations, and calls **rebalance** or **withdrawAndBridge** as the executor.

- A **relayer** that observes CCTP burns originating from `selfBatchDeposit` and `withdrawAndBridge`, retrieves the attestation from Circle, and calls `relayAndDeposit` on the destination chain.
- A **frontend** that computes performance fee amounts for `selfBatchWithdraw` and constructs the corresponding user transactions.

Security Model and Trust Assumptions

The security model rests on four assumptions: user custody of vault shares and aTokens is preserved across all flows, the three on-chain roles (owner, executor, relayer) behave honestly within the bounds described in the project documentation, the contract address is identical on every CCTP-supported chain via CreateX CREATE3, and external components (CCTP and the underlying vaults) behave as specified.

The following trust assumptions are treated as in-scope:

- **Owner honesty.** The owner is expected to be a multisig responsible for managing the vault whitelist, sweeping accumulated fee shares, and assigning the executor and relayer roles. The audit treats the owner as fully trusted.
- **Executor honesty.** The executor is an off-chain service that calls `rebalance` and `withdrawAndBridge`. It selects target vaults, fee amounts, and (in the current code) the CCTP `mintRecipient` on destination chains. The audit treats the executor as fully trusted within the bounds described in the README and `docs/laymans.md`.
- **Relayer honesty.** The relayer is an off-chain service that calls `relayAndDeposit` on destination chains. It selects the address that receives vault shares on the destination chain. The audit treats the relayer as fully trusted within the bounds described in the README and `docs/laymans.md`.
- **Performance fee integrity.** The contract enforces no bound on performance fee amounts. In the executor-callable flows (`rebalance` and `withdrawAndBridge`), the executor chooses the fee and can set it arbitrarily high, up to the user's full vault-share balance. In `selfBatchWithdraw`, the caller chooses the fee and can set it to zero. The documented 15% performance fee is a convention enforced only by the executor and the frontend, never by the contract. The audit treats correct fee selection as part of executor honesty for the executor-callable flows and as a revenue-loss risk rather than a user-loss risk for `selfBatchWithdraw`.

- **CCTP and Circle's attestation service.** The contract delegates message verification, attestation, and USDC mint/burn authority to Circle's CCTP v2 contracts. Any compromise of Circle's attestation service or of their on-chain message transmitters is out of scope.
- **Whitelisted vault integrity.** Morpho vaults and the Aave V3 Pool are assumed to behave according to their respective standards, with adequate total value locked and OpenZeppelin's `ERC4626` virtual-shares offset (or an equivalent inflation defense) where applicable. The audit does not reassess vault solvency, the underlying lending markets, or share-price manipulation against vaults that are vetted by the owner before whitelisting. The risk of an inflation attack against a freshly whitelisted, low-TVL or unprotected ERC-4626 implementation is operational and falls on the whitelist process rather than on the contract.
- **Aave performance fee path.** The contract collects performance fees on Morpho positions via `_collectFees` in the executor-callable flows and on Aave positions inline in `selfBatchWithdraw` (where fee shares are keyed by the immutable `aUsdc` address rather than `aavePool`). The executor cannot collect Aave-side performance fees in `rebalance` or `withdrawAndBridge`. This is the documented and is treated as out-of-scope for this audit.
- **Deterministic cross-chain address.** Cross-chain functionality relies on `QuicknodeEarn` deploying to the same address on every CCTP-supported chain via CreateX CREATE3. Any divergence on any chain invalidates the assumptions underlying the recommendations in this report.

Any deviation of the implementation from the trust boundaries described in the README and `docs/laymans.md` is treated as an issue in this report.

Low Severity

L-01 Owner Can Drain User Principal Despite Documented Trust Model

The README's Security Model section states that "No role can withdraw funds to an arbitrary address — all paths route back to the user or an approved vault", and [docs/laymans.md](#) frames a compromised owner as being unable to force user funds anywhere because doing so "requires either the executor (for rebalances) or the user themselves (for deposits)". Both descriptions understate the owner's actual power. In either deployed variant, a compromised owner key can drain any user with active ERC-20 approvals to the contract.

Two-step drain via `setExecutor`. The owner can assign a new executor without delay through `setExecutor`. A compromised owner installs themselves (or any attacker-controlled address) as the executor and then exercises any executor-callable function on a user that has approved the contract. Because `_collectFees` accepts caller-supplied `feeAmounts[]` with no upper bound, the attacker-executor can transfer the user's entire vault-share balance to the contract as a "fee" in a single `rebalance` or `withdrawAndBridge` call, and the owner then drains those shares with `sweep(token, amount)`. This path remains exploitable even after the other recommendations in this report are applied, because performance fees are accepted as an off-chain convention rather than enforced on-chain.

One-step drain via proxy upgrade. The `QuicknodeEarnProxy` variant exposes `_authorizeUpgrade` to the owner. A single `upgradeToAndCall` transaction can replace the implementation with one that performs unrestricted `transferFrom` calls against every address with a live approval, bypassing every role separation, vault whitelist, and parameter constraint in the contract.

Consider gating the owner actions behind a timelock contract owned by the multisig. A sufficient delay gives users time to revoke approvals in response to a compromised owner key. Update the README's Security Model section and [docs/laymans.md](#) to describe the actual trust surface: a compromised owner can, after the timelock window expires, drain any user with active approvals.

Update: Partially Resolved at commit [6a7d4fc](#). The team stated:

Acknowledged. The owner is a fully trusted role per the audit's Security Model, and we will deploy ownership behind a timelocked multisig so role reassignments and proxy upgrades are observable with delay sufficient for users to revoke approvals. Updated `README.md` and `docs/laymans.md` to disclose the actual trust surface:

- Replaced "no role can withdraw funds to an arbitrary address" with an explicit description of owner power.*
- Documented both drain paths: (1) `setExecutor` + unbounded `feeAmounts[]` + `sweep`, and (2) `upgradeToAndCall` on `QuicknodeEarnProxy`.*
- Listed "compromised owner key" as item #1 under "What the contract does NOT protect against," with the timelock as the recommended mitigation.*

L-02 Unconstrained `mintRecipient` And `destinationCaller` At CCTP Burn Entry Points

CCTP v2 burns are parameterized by two destination-chain access fields. `mintRecipient` determines which address receives the minted USDC on the destination chain, and `destinationCaller` determines which address is permitted to call `receiveMessage` on the destination chain to trigger that mint; setting `destinationCaller` to a specific address restricts the relay to that address alone, while setting it to zero allows any address to relay the message. The two fields must be chosen as a pair. If `mintRecipient` is a user wallet but `destinationCaller` is the contract on the destination chain, only the contract can trigger `receiveMessage`, yet the contract's relay path expects to deposit into a vault rather than mint to a wallet, leaving the bridged USDC unreachable. If `mintRecipient` is the contract but `destinationCaller` is zero, anyone can relay the message on the destination chain, removing the MEV protection the contract relies on for atomic relay+deposit.

Three entry points perform CCTP burns: `withdrawAndBridge` (executor-only), `selfBatchDeposit` (user-callable), and `selfBatchWithdraw` (user-callable). In all three, both fields are forwarded from caller-supplied arguments into `__cctpBurn` and on to `ITokenMessengerV2.depositForBurn`, with no constraint beyond `mintRecipient` being non-zero. Only a small set of `(mintRecipient, destinationCaller)` pairings is consistent with the intended use of each entry point. In `withdrawAndBridge`, two pairings are valid: for a cross-chain rebalance, both fields must be the contract's address on the destination chain so the relayer can complete the deposit via `relayAndDeposit` under MEV protection; for a cross-chain exit, `mintRecipient` must be the `user` parameter and `destinationCaller` must be zero so the burn can be relayed permissionlessly to the user's

wallet. In `selfBatchDeposit`, two pairings mirror the same structure with `msg.sender` substituting for `user`. In `selfBatchWithdraw`, only one pairing is valid: a permissionless bridge-back with `mintRecipient = msg.sender` and `destinationCaller = 0`.

The threat models differ across entry points. In `withdrawAndBridge`, the caller is the executor, who acts on vault shares the user has pre-approved the contract to spend for the specific purpose of yield optimization. A compromised executor can set `mintRecipient` to any address on any destination chain and steal the user's principal, contradicting the README's Security Model claim that no role can withdraw funds to an arbitrary address. In `selfBatchDeposit` and `selfBatchWithdraw`, the caller is the user themselves: a redirection of the burn requires the user to sign a malicious transaction. The input validation in those two entry points therefore serves primarily to prevent stranded burns and accidental misuse rather than to defend against an external attacker.

Consider constraining both fields at every entry point and reverting on any unexpected pairing:

- In `withdrawAndBridge`, allow `mintRecipient = bytes32(uint256(uint160(address(this))))` and `destinationCaller = bytes32(uint256(uint160(address(this))))` for cross-chain rebalance, and `mintRecipient = bytes32(uint256(uint160(user)))` and `destinationCaller = bytes32(0)` for cross-chain exit.
- In `selfBatchDeposit`, allow `mintRecipient = bytes32(uint256(uint160(address(this))))` and `destinationCaller = bytes32(uint256(uint160(address(this))))` for a vault deposit on the destination chain, and `mintRecipient = bytes32(uint256(uint160(msg.sender)))` and `destinationCaller = bytes32(0)` for a direct mint back to the caller's wallet.
- In `selfBatchWithdraw`, allow only `mintRecipient = bytes32(uint256(uint160(msg.sender)))` and `destinationCaller = bytes32(0)`.

All cases rely on CreateX CREATE3 deployment parity so that `address(this)` resolves to the deployed contract on every CCTP-supported destination chain.

Update: Resolved at commit [754ff51](#). The team stated:

Constrained the `(mintRecipient, destinationCaller)` pair at every CCTP burn entry point per the audit's recommendation. Any unexpected pairing reverts with `InvalidInput()`.

- `withdrawAndBridge` (executor):

- `(address(this), address(this))` — MEV-protected cross-chain rebalance via `relayAndDeposit`.
- `(user, bytes32(0))` — permissionless cross-chain exit; anyone can call `receiveMessage` on the destination chain to mint USDC directly to the user's wallet.
- `selfBatchDeposit (user)`:
- `(address(this), address(this))` per burn entry — MEV-protected vault deposit on the destination chain.
- `(msg.sender, bytes32(0))` per burn entry — permissionless mint back to caller's wallet.
- `selfBatchWithdraw (user)`:
- `(msg.sender, bytes32(0))` per burn entry — permissionless bridge-back to the caller's wallet on the source chain.

The `(user, 0)` and `(msg.sender, 0)` pairings give users a built-in fallback for relay downtime: the burn is permissionless on the destination chain, so the user (or anyone on their behalf) can call `receiveMessage` directly without contract assistance. All cases rely on the documented CreateX CREATE3 deployment parity that makes `address(this)` resolve to the deployed contract on every CCTP-supported chain.

L-03 Relayer Can Redirect Bridged Deposits To An Arbitrary Beneficiary

The `relayAndDeposit` function is the relayer-only entry point that finalizes a cross-chain deposit on the destination chain. It calls `IMessageTransmitter.receiveMessage` with the CCTP `message` and `attestation`, measures the USDC minted to the contract via a balance delta, and deposits that USDC into the specified vaults on behalf of the `user` parameter.

The `user` parameter is never validated against the content of `message`. CCTP v2 messages in the basic `depositForBurn` flow do not encode a beneficiary distinct from the `mintRecipient`, so the contract has no on-chain source of truth for the intended recipient of the resulting vault shares. A compromised relayer can therefore relay a legitimate user's attestation while passing any address as `user`, directing the minted vault shares to a beneficiary of the relayer's choosing. The original depositor burned USDC on the source chain and receives nothing on the destination chain. The README states that "a compromised

relayer can only deposit bridged USDC into whitelisted vaults", which, while technically correct at the vault level, omits that the beneficiary of those shares is fully relayer-controlled.

Consider switching the source-chain burn in `selfBatchDeposit` from `depositForBurn` to `depositForBurnWithHook`, encoding the intended beneficiary in `hookData`. On the destination chain, parse `hookData` from the relayed message and require that the `user` parameter passed to `relayAndDeposit` matches the decoded beneficiary. This removes the relayer's ability to redirect bridged deposits and aligns the implementation with the README's trust model.

Update: Resolved at commit [3e9359c](#). The team stated:

Switched all CCTP burns from `depositForBurn` to `depositForBurnWithHook`, encoding the intended beneficiary as `hookData`. `relayAndDeposit` now parses the beneficiary from the signed CCTP message at byte offset 376 (148-byte header + 228-byte burn body) and reverts with `InvalidUser()` if the `user` parameter doesn't match.

- `ITokenMessengerV2` updated to `depositForBurnWithHook` (extra bytes calldata `hookData`).

- `_cctpBurn` takes a `beneficiary` arg, ABI-encoded as `hookData`. Switched to `abi.encodeCall` for type safety.

- `withdrawAndBridge` passes `user`; `selfBatchDeposit` and `selfBatchWithdraw` pass `msg.sender`.

- `relayAndDeposit` decodes `abi.decode(message[376:408], (address))` and validates against `user`.

`hookData` is handled internally by `_cctpBurn`.

L-04 `relayAndDeposit` Strands Excess Minted USDC

The `relayAndDeposit` function relays a CCTP v2 message, snapshots the resulting USDC balance delta as `minted`, sums the relayer-supplied `amounts[]` as `totalNeeded`, and reverts if `minted < totalNeeded`. When `minted` exceeds `totalNeeded`, the function deposits each `amounts[i]` into its target vault on behalf of the user but does not return the difference. The remainder sits in the contract balance and is later sweepable by the owner via

`sweep`. This contrasts with `selfBatchWithdraw`, where any USDC left over after the configured CCTP burns is explicitly transferred back to `msg.sender`.

The asymmetry produces silent capture of user principal under honest relay behavior. CCTP charges a per-message fee, so the amount minted on the destination chain is strictly less than the amount burned on the source chain, and the relay must populate `amounts[]` to match the post-fee mint amount. Conservative fee estimation by the relay leaves a non-zero remainder. For example, if a user initiates a `selfBatchDeposit` burn for 1000 USDC and the post-fee mint is 998 USDC, a relay that sets `amounts = [995]` to leave a small buffer ends up depositing 995 into the vault and stranding 3 USDC of user principal in the contract. The owner's subsequent `sweep` removes those funds from the user's recoverable balance entirely. The relay is a trusted role, so this is honest-mistake territory rather than adversarial, but the loss is silent and irreversible from the user's perspective.

Consider transferring any remainder to the `user` parameter via `safeTransfer` after the deposit loop completes. Computing the remainder as a post-loop balance delta against the pre-relay snapshot (`usdc.balanceOf(address(this)) - before`) is more robust than the `minted - totalNeeded` alternative, because the balance delta also accounts for any non-standard vault deposit that consumes more or less than the requested `amounts[i]`. The pattern matches the contract's existing reliance on balance deltas as ground truth in place of return values from external CCTP and vault calls.

Update: Resolved at commit [92de56b](#). The team stated:

In production, we use standard transfer speeds, which cost 0 USDC per transfer, but for smart contract correctness `relayAndDeposit` now transfers any post-deposit USDC remainder to the user via a balance-delta check after the deposit loop, preventing silent capture of user principal from conservative CCTP fee estimation.

Same balance-delta-and-refund pattern was added defensively to `rebalance` (any USDC left behind by a non-standard ERC-4626 vault) and `selfBatchDeposit` (any leftover from the pulled `total`).

L-05 Incomplete Input Validation Across Multiple Functions

Several functions accept parameters without validating their consistency or non-emptiness. The downstream effects range from opaque out-of-bounds panics to silent no-ops that allow fees to be collected without any underlying vault work. Each case below represents an independent input validation gap.

Constructor does not validate `aavePool` and `aUsdc` pairing. The `constructor` accepts `_aavePool` and `_aUsdc` as independent optional parameters, with the intent that both are either set (Aave V3 available on chain) or both zero. If `_aavePool` is non-zero while `_aUsdc` is zero, any call that routes through the Aave withdrawal path attempts `IERC20(address(0)).safeTransferFrom(...)` and reverts. Because the addresses are immutable, the break is permanent for the deployment's lifetime.

Missing array length checks. Three functions accept paired arrays without verifying they have equal length. In `rebalance` and `withdrawAndBridge`, `feeVaults[]` and `feeAmounts[]` are forwarded to `_collectFees`, which iterates over `feeVaults.length` and indexes into `feeAmounts[i]` with no length check. In `relayAndDeposit`, `vaults[]` and `amounts[]` are iterated in separate loops; a divergence produces either an out-of-bounds panic or an unrelated `ZeroAmount` revert depending on which array is longer. The `selfBatchWithdraw` function uses `ArrayLengthMismatch` for the equivalent case, but the executor-facing and relayer-facing entry points do not.

`shares == 0` not rejected in `rebalance` and `withdrawAndBridge`. Neither function rejects zero-share calls. On the ERC-4626 → ERC-4626 path, `redeem(0, ...)` and `deposit(0, user)` are permitted no-ops under the standard, and both `_withdrawFromVault` and `_depositToVault` complete without work. Furthermore, `_collectFees` runs to completion before the no-op withdrawal, so fee shares transfer successfully even though no vault capital is moved. The executor can combine `shares == 0` with arbitrary `feeAmounts` to extract fee shares without touching the user's position.

`fromVault == toVault` not blocked in `rebalance`. The `rebalance` function does not reject same-vault rebalances. Combined with executor-supplied fees, a same-vault rebalance becomes a griefing primitive that collects fees without changing the user's position. [docs/laymans.md](#) claims this case is blocked; the implementation does not enforce it. Adding a same-vault check closes this specific pattern but does not eliminate the underlying griefing surface, because an executor can achieve the same effect by ping-ponging shares between two different whitelisted vaults. A complete mitigation requires bounding the fee amount on-chain, which is outside the scope of this input-validation finding.

Silent fee drop in `selfBatchWithdraw`. In `selfBatchWithdraw`, when `shares[i] == 0` the per-vault amount `amt` is resolved from `balanceOf(msg.sender)`. If that balance is also zero, the loop hits the `if (amt == 0) continue` early-exit and skips the `if (feeAmt >= amt) revert InvalidInput()` sanity check. A caller who passes `feeAmounts[i] > 0` for a zero-balance vault receives no error; the malformed fee argument is silently ignored, masking bugs in the caller's fee computation.

`_depositToVault` does not validate `sharesReceived > 0`. The `_depositToVault` helper returns the share amount minted by the destination vault but does not require it to be non-zero. A zero return on a non-zero deposit indicates that the user's position did not register and the call should fail closed rather than complete silently. The Aave branch is unaffected because `_depositToVault` hardcodes its return value to `amount`.

`shares[i] == 0` sentinel in `selfBatchWithdraw` interacts poorly with max-approval UX. The same `selfBatchWithdraw` sentinel that triggers the silent fee drop above is documented as a "withdraw the user's full balance" shortcut, resolving the per-vault amount from `IERC20(vault).balanceOf(msg.sender)` (or `IERC20(aUsdc).balanceOf(msg.sender)` on the Aave path). One-time max-approval is the conventional UX for ERC-4626 share tokens, so users typically have signed `approve(this, type(uint256).max)` on every vault they hold a position in. A subsequent `selfBatchWithdraw` call where `shares[i]` is zero for any reason (a frontend bug, a copy-paste, a malicious or compromised frontend, or a user reading zero as "no-op") drains the entire balance of that vault into the contract. The contract cannot distinguish "withdraw nothing" from "withdraw everything" once the sentinel is in scope.

Consider adding the following checks:

- In the constructor, require `( _aavePool == address(0) ) == ( _aUsdc == address(0) )` and revert with `ZeroAddress()` otherwise.
- In `rebalance`, `withdrawAndBridge`, and `relayAndDeposit`, revert with `ArrayLengthMismatch()` when the paired arrays diverge.
- In `rebalance` and `withdrawAndBridge`, revert with `ZeroAmount()` when `shares == 0`.
- In `rebalance`, revert with `InvalidInput()` when `fromVault == toVault`.
- In `selfBatchWithdraw`, drop the `(0 = full balance)` sentinel entirely and revert with `ZeroAmount()` when any `shares[i] == 0`. Frontends and direct callers that want to close a full position can read the user's vault-share balance off-chain and pass it explicitly. This change subsumes the silent fee drop above (the early-continue path becomes unreachable) and removes the max-approval footgun in a single edit.
- In `_depositToVault`, on the ERC-4626 branch, revert with `ZeroAmount()` when the vault returns `sharesReceived == 0`. The Aave branch is unchanged.

Update: Resolved at commit [344ff7a](#). The team stated:

Added input validation guards across executor and relay entry points:

- `rebalance`: rejects `shares == 0`, `fromVault == toVault`, and `feeVaults.length != feeAmounts.length`.

- `withdrawAndBridge`: rejects `shares == 0` and `feeVaults.length != feeAmounts.length`.
- `relayAndDeposit`: rejects `vaults.length != amounts.length`.
- `selfBatchWithdraw`: dropped the `shares[i] == 0` → full balance sentinel; zero entries now revert with `ZeroAmount()`. Closes the max-approval footgun and the silent-fee-drop edge case in one edit.
- `_depositToVault`: reverts with `ZeroAmount()` when an ERC-4626 vault returns `sharesReceived == 0` on a non-zero deposit.

Skipped the `aavePool/aUsdc` constructor check — Aave integration has been removed from the contract entirely.

Tests in `test/QuicknodeEarn.test.ts` updated for the removed sentinel.

L-06 Vault De-Whitelisting Blocks Operations On Existing User Positions

The `rebalance`, `withdrawAndBridge`, and `selfBatchWithdraw` functions each require their source-side vault to be in the `approvedVaults` whitelist and revert otherwise. The owner can remove a vault at any time via `removeVault`, which clears the approval flag immediately and without delay.

After a vault is removed, all three source-side operations against user positions held in it revert. The executor can no longer migrate the user out via `rebalance` or bridge them out via `withdrawAndBridge`. The user can no longer self-exit through `selfBatchWithdraw`, including the cross-chain bridge-back path enabled by the `burns[]` parameter. Custody is preserved because the shares remain in the user's wallet, and the user retains the option of redeeming directly against the underlying ERC-4626 vault or Aave Pool, but the contract's full set of contract-mediated paths becomes unavailable for that vault.

Consider removing the source-side `approvedVaults` check from all three functions. The check is redundant given that pulling shares from a user already requires the user to have granted the contract an allowance on that vault's share token (per [docs/laymans.md](#) L42), which serves as the consent signal under which a privileged role may operate on the position. The whitelist's stated purpose at [docs/laymans.md](#) L88-92 is to control where funds flow to, not where they flow from. The destination-side check on `rebalance.toVault` should

remain intact, so that owner-initiated de-whitelisting continues to bound new deposits without trapping existing user positions.

Update: Resolved at commit [f319fbb](#). The team stated:

Removed source-side `approvedVaults` checks from `rebalance`, `withdrawAndBridge`, and `selfBatchWithdraw`. The whitelist now only gates destination vaults (where funds flow into), so `removeVault` no longer traps existing user positions. Users and the executor can always exit via contract-mediated paths regardless of delisting.

The destination-side check in `rebalance approvedVaults[toVault]` is preserved so owner-initiated de-whitelisting continues to bound new deposits.

L-07 Executor And Relayer Can Deposit On A User's Behalf Without Their Share Approval

The `rebalance` function (executor-only) and the `relayAndDeposit` function (relayer-only) deposit USDC into vaults on the user's behalf via `_depositToVault`. For ERC-4626 vaults this calls `deposit(amount, user)`, and for the Aave V3 Pool it calls `supply(asset, amount, user, ...)`; in both cases the resulting vault shares or aTokens mint directly to the user's wallet. Neither path consumes any user-side allowance, and neither function checks that the user has approved the target vault's share token to the contract.

[docs/laymans.md](#) L42 lists "operate without the user's prior ERC20 approval" among the operations the executor cannot perform, framing user approvals as a constraint on what privileged roles can do. The constraint holds at the withdraw leg, where pulling shares out of `fromVault` requires the user to have approved the contract for that vault's share token. It does not hold at the deposit leg, where the privileged role can target any whitelisted vault regardless of whether the user has approved it. A relayer or executor can therefore place shares in a vault the user has never opted into. Custody is preserved because the shares are in the user's wallet, so this is not a fund-loss path, but the resulting position is stranded from the contract's management: subsequent contract-mediated rebalances, exits, or bridge-outs against those shares will revert at `safeTransferFrom` until the user grants the missing approval. The user must either approve the unwanted vault's share token to recover contract-mediated paths, or redeem directly against the vault outside of the contract.

Consider checking `IERC20(targetVault).allowance(user, address(this)) > 0` (or the corresponding `aUSDC` allowance for Aave deposits) at the top of `rebalance` and `relayAndDeposit`, and reverting if the user has not granted approval on the target vault.

This brings the deposit leg into line with the withdraw leg's existing constraint and ensures the user's approval acts as a symmetric signal of which vaults privileged roles may use on their behalf. The check introduces a new failure mode in `relayAndDeposit`: if the user has not approved any whitelisted vault on the destination chain when the relayer arrives, every choice of `vaults[]` reverts and the bridged CCTP message remains unconsumed indefinitely. Consider pairing the constraint with a destination-chain escape path that the user (or a trusted role acting on a user-validated request) can invoke to consume the message via `receiveMessage` and either deliver the minted USDC to the user's wallet or burn it back to the source chain, so users are never trapped with un-relayable bridged funds.

Update: Resolved at commit [d072a12](#). The team stated:

Added share-token allowance checks on the deposit leg of `rebalance toVault` and `relayAndDeposit` (each destination vault), ensuring privileged roles can only deposit into vaults the user has explicitly opted into.

To address the new failure mode this introduces for in-flight CCTP messages — bridged USDC stranded if the user has no destination-vault allowance — added `emergencyClaimBridge`, a destination-chain escape hatch.

How it works:

The function consumes a pending CCTP V2 message via `receiveMessage` and forwards the minted USDC to the hookData beneficiary's wallet, bypassing the vault deposit entirely. Authorization is via the hookData beneficiary committed at burn time (encoded by `_cctpBurn`, validated by `relayAndDeposit`). Only callable by that beneficiary on the destination chain.

When it applies:

This hatch is needed only for the `(this, this)` MEV-protected pairing. The contract is the `destinationCaller`, so without an in-contract path a stalled message would be unreachable. The `(user, 0)` and `(msg.sender, 0)` pairings allowed by bug #3 don't need this hatch since `destinationCaller == 0` lets anyone (including the user) call `receiveMessage` directly.

L-08 Documentation Inconsistent With Implementation

Several sections of the project documentation describe behavior that the implementation does not exhibit, or obscure trust dependencies that the implementation relies on. Each case below is independent and affects a different reader audience.

Self-serve framing in the README understates relay dependency for cross-chain deposits. The [README](#) states that users can create strategies via [selfBatchDeposit](#) and close them via [selfBatchWithdraw](#) "without owner involvement, providing a trustless exit path independent of the rebalancer service". The exit path through [selfBatchWithdraw](#) is free of privileged-role liveness dependencies in the sense that no executor or relay action is required to complete a user's exit; the user signs and submits the transaction themselves. The entry path through [selfBatchDeposit](#) with cross-chain burns is not, when used as intended. The intended use sets [mintRecipient](#) to the contract on the destination chain so the relay can call [relayAndDeposit](#) and deposit the minted USDC into a vault on the user's behalf. That destination-chain deposit is gated behind [onlyRelayer](#): if the relay is offline, the attestation remains unclaimed and the user's bridged USDC cannot be deposited into the intended vault. The case where [mintRecipient](#) is set to the user's own wallet reduces to a plain cross-chain CCTP mint and completes without the project's relay, but this is not the intended vault-deposit flow.

[fromVault != toVault](#) check described as "explicitly blocked" in [docs/laymans.md](#). Both the executor-constraints section at L41 and the protective-mechanisms table at L135 of [docs/laymans.md](#) assert that same-vault rebalances are blocked by the contract. No such check exists in [rebalance](#). Regardless of whether the code is updated to add the check, the documentation must match the implemented behavior.

The executor's cross-chain rebalance flow is not documented. Neither the README nor [docs/laymans.md](#) names the intended executor-initiated cross-chain rebalance flow, in which the executor calls [withdrawAndBridge](#) on chain A with [mintRecipient](#) and [destinationCaller](#) both set to the contract on chain B, after which the relay calls [relayAndDeposit](#) on chain B to deposit into the target vault. The existence of this flow is only implied, by the [withdrawAndBridge NatSpec](#) and by [docs/laymans.md](#) L39. Without the flow being named, a reader cannot reconstruct from the public specification why [mintRecipient](#) and [destinationCaller](#) must be constrained to the contract's own address on the destination chain, nor the purpose served by the CreateX CREATE3 deployment parity that the constraint depends on.

Executor's "cannot operate without user approval" claim in [docs/laymans.md](#) is one-sided. The executor-constraints list at L42 of [docs/laymans.md](#) states that the executor cannot operate without the user's prior ERC-20 approval, on the basis that the contract uses [safeTransferFrom](#). The claim holds for operations that pull shares out of a user's position, but the deposit leg of [rebalance](#) and the deposit performed by [relayAndDeposit](#) do not consume any user-side allowance. The executor and relayer can therefore deposit on the user's behalf into a whitelisted vault the user has not approved. The doc statement should specify that the approval-based constraint applies only to operations that pull funds from the user, not to deposits placed on the user's behalf.

[docs/deployment.md](#) describes a UUPS workflow that does not match [Deploy.s.sol](#). [docs/deployment.md](#) describes a deployment architecture built around an ERC-1967 proxy, a separate upgradeable implementation, an [initialize\(owner, vaults\)](#) call, and upgrades via [upgradeToAndCall](#). The [forge verify-contract](#) examples target [contracts/QuicknodeEarnProxy.sol:QuicknodeEarnProxy](#). The [Deploy.s.sol](#) script deploys [QuicknodeEarn.sol](#) (the non-proxy variant) directly via CreateX CREATE3, with no proxy, no [initialize](#), and no implementation separation. An operator following [docs/deployment.md](#) to upgrade the currently deployed contract would attempt to call [upgradeToAndCall](#) on a contract that has no such function.

Consider the following updates:

- Revise the README's overview to distinguish the trust profiles of [selfBatchDeposit](#) (cross-chain deposits depend on the relayer until an escape hatch is implemented) and [selfBatchWithdraw](#) (genuinely independent of privileged roles).
- Remove or correct the two statements in [docs/laymans.md](#) that describe a same-vault rebalance check, and re-align with the implementation (or the fix applied during remediation).
- Reword the executor-constraints entry at [docs/laymans.md](#) L42 to state that the user-approval requirement applies to operations that pull shares from the user, and to acknowledge that deposit-side operations on the user's behalf are bounded by the vault whitelist alone.
- Either split [docs/deployment.md](#) into clearly labeled sections for the non-proxy ([QuicknodeEarn.sol](#)) and proxy ([QuicknodeEarnProxy.sol](#)) variants, or replace the document's contents with the CreateX CREATE3 flow used by [Deploy.s.sol](#) and mark the proxy flow as not currently active.
- Document the executor-initiated cross-chain rebalance flow: [withdrawAndBridge](#) with [mintRecipient = address\(this\)](#) and [destinationCaller = address\(this\)](#), followed by [relayAndDeposit](#) on the destination chain. This provides the public rationale for why [mintRecipient](#) and [destinationCaller](#) are

expected to point at the contract's own address on the destination chain, and why CreateX CREATE3 deployment parity is a trust assumption of the system.

Update: Resolved at commit [69bae8e](#). The team stated:

Updated `README.md`, `docs/laymans.md`, and `docs/deployment.md` to align with the implementation:

- `selfBatchDeposit` trust profile: README's "trustless exit path independent of the rebalancer" framing rewritten to distinguish `selfBatchWithdraw` (genuinely independent of privileged roles) from `selfBatchDeposit` cross-chain burns (the `(this, this)` pairing depends on the relayer; the `(msg.sender, 0)` pairing does not). The new `emergencyClaimBridge` destination-chain escape (bug #8) is documented as the recovery path for stalled `(this, this)` burns.
- Same-vault rebalance: `docs/laymans.md` no longer claims same-vault rebalances are blocked at L41/L135 in error — the bug #6 fix actually enforces `fromVault != toVault` in `rebalance`, so the docs now match the implementation.
- Executor cross-chain flows: added a named description in `README.md` covering both pairings — `(this, this)` for the MEV-protected rebalance flow (executor + relayer two-step) and `(user, 0)` for the permissionless cross-chain exit (anyone can call `receiveMessage` on the destination chain). Explicitly calls out CreateX CREATE3 deployment parity as the trust assumption that makes `address(this)` a valid stand-in across chains.
- Executor approval claim: `docs/laymans.md` reworded to make clear the user-approval requirement applies to both legs (withdraw via `safeTransferFrom`, deposit via the bug #8 allowance check on `toVault` / destination vaults), not just the withdraw leg.
- Source-side whitelist: `docs/laymans.md` reworded to reflect the bug #7 change — `toVault` is the only checked side, so positions can always be exited even if a vault is later delisted.
- Removed stale `shares[i] == 0` warning: `docs/laymans.md` no longer warns about a zero-sentinel over-withdrawal footgun — bug #6 removed that sentinel; zero now reverts with `ZeroAmount()`.
- Deployment doc: `docs/deployment.md` corrected — `deployProxy(uint32)` / `deployProxyWithVaults` are the proxy entry points; `run(uint32)` / `runWithVaults(uint32)` are legacy non-proxy entries that should not be used for

new deployments. Subsequent upgrades to a proxy deployment go through `executeUpgrade(uint32)`.

L-09 Proxy Variant Does Not Use Upgradeable OpenZeppelin Or ERC-7201 Namespaces

`QuicknodeEarnProxy` inherits from `Ownable2Step` and `ReentrancyGuard` from `@openzeppelin/contracts/access/` and `@openzeppelin/contracts/utils/`, alongside `Initializable` and `UUPSUpgradeable` from `@openzeppelin/contracts/proxy/utils/`. The pinned dependency version is OpenZeppelin Contracts v5.0.0 (per the `lib/openzeppelin-contracts` submodule). Of these parents, `Initializable` uses ERC-7201 namespaced storage and `UUPSUpgradeable` declares no storage of its own (only an `immutable` and a `constant`). `Ownable`, `Ownable2Step`, and `ReentrancyGuard` are not namespaced and occupy sequential slots: `__owner` at slot 0, `__pendingOwner` at slot 1, and `__status` at slot 2. The contract's own state variables (`approvedVaults`, `vaultList`, `executor`, `relayer`) follow at slots 3 onward, also without an ERC-7201 namespace and without a storage gap.

`forge inspect QuicknodeEarnProxy storage-layout` confirms this exact layout.

The current layout is internally consistent, but it is structurally fragile under upgrade. If a future OpenZeppelin release — or a planned OpenZeppelin upgrade to v5.1.0+ where `ReentrancyGuard` was migrated to ERC-7201 — adds, removes, or relocates a state variable in `Ownable`, `Ownable2Step`, or `ReentrancyGuard`, every state slot in `QuicknodeEarnProxy` shifts by the inserted width on the next implementation rebuild, silently corrupting the vault whitelist, the vault enumeration list, and the executor and relayer addresses. The same corruption follows from any upgrade that reorders or extends the inheritance list. The owner's authority to authorize an upgrade is unchecked by any storage-layout test in the repository, so a bad upgrade would not be caught before it lands on chain.

Consider switching `QuicknodeEarnProxy` to inherit `Ownable2StepUpgradeable` and `ReentrancyGuardUpgradeable` from `@openzeppelin/contracts-upgradeable/` (which use ERC-7201 namespaced storage and expose `__Ownable_init` / `__Ownable2Step_init` / `__ReentrancyGuard_init` initializers), and applying ERC-7201 to the contract's own state so that `approvedVaults`, `vaultList`, `executor`, and `relayer` live at a derived namespace slot rather than sequential storage. With both parents and the contract's own state on namespaced layouts, inheritance order and parent state additions are no longer load-bearing for upgrade safety. Pairing the change with an

OpenZeppelin Upgrades plugin storage-layout check in CI ensures any future implementation that breaks layout compatibility fails before deployment.

Update: Resolved at commit [e3d5072](#). The team stated:

Migrated `QuicknodeEarnProxy` to ERC-7201 namespaced storage. All mutable state now lives in a single `EarnStorage` struct `approvedVaults`, `vaultList`, `executor`, `relayer`) at the slot derived from the namespace `quicknode.earn.storage`. `forge inspect QuicknodeEarnProxy storage-layout` reports zero sequential slots.

- Inherits `Ownable2StepUpgradeable` from `@openzeppelin/contracts-upgradeable`. `ReentrancyGuard` is also ERC-7201 namespaced in OZ v5.5+ and is replaced by `ReentrancyGuardTransient` in bug #11.

- Implementation constructor calls `_disableInitializers()`; `_authorizeUpgrade(address)` is `onlyOwner`.

- Public state vars replaced with explicit `executor()`, `relayer()`, `getApprovedVaults()`, `isVaultApproved(address)` getters.

- Added `@openzeppelin/contracts-upgradeable@^5.6.1` to `package.json`, matching remapping in `foundry.toml`. Tests in `test/QuicknodeEarn.test.ts` exercise the upgrade flow and assert state persistence across `upgradeToAndCall`.

- CI workflow at `.github/workflows/storage-layout.yml` enforces three checks on every PR: zero sequential slots, `EARN_STORAGE_LOCATION` matches the keccak derivation of `quicknode.earn.storage`, and the `EarnStorage` struct definition matches a committed snapshot at `.github/storage-snapshots/EarnStorage.struct` (catches reorders/retypes that `forge inspect` cannot see).

L-10 `abi.encodeWithSelector` Used Instead Of Type-Safe `abi.encodeCall`

The `_cctpBurn` function constructs calldata for `ITokenMessengerV2.depositForBurn` via `abi.encodeWithSelector`. This encoder accepts arbitrary argument types and counts with no compile-time verification against the target interface. A future change to the `depositForBurn` signature, a typo in argument order, or a divergence between the local interface declaration and the encoded arguments would compile without warning and produce calldata that does not match the real function.

Consider replacing the call with `abi.encodeCall`, which has been available since Solidity 0.8.11 and verifies each argument's type against the interface signature at compile time, so any mismatch between the call site and the declared interface becomes a compile error rather than a silent runtime mismatch.

Update: Resolved at commit [3e9359c](#).

L-11 OpenZeppelin Dependency Version Skew Between Hardhat And Foundry

The project resolves `@openzeppelin/contracts` to two different versions depending on the toolchain. Hardhat uses Node module resolution and reads the npm package pinned at `package.json`, which requests `^5.6.1`. Foundry applies the remapping declared in `foundry.toml` and resolves the same import path against the `lib/openzeppelin-contracts` submodule, which is pinned at v5.0.0. The two dependencies are not interchangeable: between v5.0.0 and v5.6.1 OpenZeppelin migrated `ReentrancyGuard` from a sequential `_status` slot to an ERC-7201 namespace, and made smaller adjustments to other utilities. As a result, the bytecode produced by `hardhat compile` for `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` differs from the bytecode produced by `forge build` against the same source.

The Hardhat test suite at `test/QuicknodeEarn.test.ts` exercises the v5.6.1-linked artifacts, while the Foundry deploy script at `script/Deploy.s.sol` ships the v5.0.0-linked artifacts. Tests therefore validate behavior and storage layout that are not the ones reaching the chain. For `QuicknodeEarnProxy`, the divergence also changes the storage layout: with v5.6.1 `ReentrancyGuard` is namespaced and the contract's own state begins at slot 2, with v5.0.0 it begins at slot 3 because `_status` still occupies a sequential slot. Any future test that asserts on slot positions, any storage-layout regression check, and any reasoning that takes the test-time layout as authoritative will diverge from the deployed contract.

Consider aligning the two toolchains on a single OpenZeppelin version. The most direct fix is to update the `lib/openzeppelin-contracts` submodule to the v5.6.1 tag and bump the pin in `package.json` to a matching exact version (`5.6.1` rather than `^5.6.1`), so that both Hardhat and Foundry compile against identical sources. A CI check that compares the keccak of the runtime bytecode emitted by `hardhat compile` and `forge build` for each in-scope contract would prevent the skew from reappearing silently on a future dependency bump.

Update: Resolved at commit [55cb42a](#). The team stated:

Removed the `lib/openzeppelin-contracts` git submodule and remapped Foundry to resolve `@openzeppelin/contracts` from `node_modules/`, so both Hardhat and Forge compile against the same npm-installed v5.6.1. `package.json` is pinned to exact 5.6.1 (no caret). Added a CI bytecode-parity workflow at `.github/workflows/bytecode-parity.yml` that compares the keccak of hardhat compile vs forge build runtime bytecode for each in-scope contract on every PR, preventing the skew from reappearing silently on a future dependency bump.

Notes & Additional Information

N-01 Use `ReentrancyGuardTransient` For Per-Call Gas Savings

Both `QuicknodeEarn` and `QuicknodeEarnProxy` inherit `ReentrancyGuard` from `@openzeppelin/contracts/utils/`. Each `nonReentrant`-protected entry point (`rebalance`, `withdrawAndBridge`, `relayAndDeposit`, `selfBatchDeposit`, `selfBatchWithdraw`, and `sweep`) pays an SSTORE on entry to flip `_status` from `NOT_ENTERED` to `ENTERED`, and a second SSTORE on exit to reset it. The same protection can be implemented with TSTORE/TLOAD at a fraction of the cost on chains that support EIP-1153 transient storage.

Consider switching both contracts to `ReentrancyGuardTransient` from `@openzeppelin/contracts/utils/` on deployment targets where EIP-1153 is available. The API (`nonReentrant` modifier, `_reentrancyGuardEntered()` helper) is identical, so call sites do not change. Per-target EIP-1153 support should be confirmed before adopting the transient variant on any given chain. As a tangential note, the storage-based `ReentrancyGuard` in recent OpenZeppelin releases carries a deprecation notice scheduled for removal in OpenZeppelin v6.0, but with the project pinning OpenZeppelin to a fixed version this is informational rather than a forcing function.

Update: Resolved at commit [0a9c4c7](#). The team stated:

Resolved. Both `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` now inherit `ReentrancyGuardTransient` instead of `ReentrancyGuard`. All 7 deployment targets (Ethereum, Optimism, Unichain, Polygon, Monad, Base, Arbitrum) support

EIP-1153. Foundry config now explicitly sets `evm_version = "cancun"` (Hardhat already had this).

N-02 Redundant `forceApprove` Calls In CCTP Burn Loop

The `_cctpBurn` helper issues `usdc.forceApprove(tokenMessenger, amount)` on every invocation. When `selfBatchDeposit` and `selfBatchWithdraw` call `_cctpBurn` inside a loop over `BridgeBurn[]`, this issues one approval per burn entry to the same `tokenMessenger` address, producing N approvals for N burns.

Consider summing the total burn amount at the call site, issuing a single `usdc.forceApprove(tokenMessenger, burnTotal)` before the loop, and removing the per-call approval from `_cctpBurn`. Callers that already aggregate `localTotal` and `burnTotal` separately can reuse the existing total.

When migrating to the single pre-loop approval, also reset the allowance to zero after the loop in `selfBatchWithdraw`. The pre-loop approval there is naturally sized to `netUsdc` (the contract's USDC delta after the redeem leg), but the actual sum of burned amounts may be smaller when the user splits the close across multiple `burns[]` entries without using the `type(uint256).max` sentinel. In that case the remainder is returned to the user via `safeTransfer` and the leftover allowance to `tokenMessenger` lingers. Issuing `usdc.forceApprove(tokenMessenger, 0)` after the burn loop closes the residual cleanly. The same pattern is unnecessary in `selfBatchDeposit` because the pre-loop approval there equals the exact sum of `burns[i].amount` consumed inside the loop.

Update: Resolved at commit [5d997d3](#). The team stated:

Resolved. Removed the per-call `forceApprove` from `_cctpBurn` and moved approval to each call site:

- `selfBatchDeposit`: Split the validation into `localTotal` and `burnTotal`. Single approval for `burnTotal` before the burn loop. No cleanup needed since the full amount is consumed.*
- `selfBatchWithdraw`: Single approval for `netUsdc` before the burn loop, then zeroed after since `netUsdc` can exceed the actual sum burned when the user doesn't use the `type(uint256).max` sentinel.*

- `withdrawAndBridge`: Single approval before the one `_cctpBurn` call. Exact amount, no cleanup needed.

Same changes in both `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol`.

N-03 Code Improvements

The contracts contain several minor code-quality items that do not affect correctness but are worth addressing before deployment.

Floating pragma. Both `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` declare `pragma solidity ^0.8.24;`. A caret pragma allows compilation against any future patch or minor version in the `0.8.x` range, which can lead to subtle differences between the version used during audit and the version used at deployment. Consider pinning to a single fixed version such as `pragma solidity 0.8.24;` for production deployment.

Non-explicit imports. Every `import` statement in `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` imports the entire module without naming the symbols actually used. This pollutes the namespace and makes it harder to see at a glance which symbols a file depends on. Consider switching to named imports of the form `import {Ownable2Step} from "@openzeppelin/contracts/access/Ownable2Step.sol";`.

Unnamed mapping parameters. The `approvedVaults` mapping in `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` is declared as `mapping(address => bool)`. Solidity 0.8.18 introduced named mapping parameters, which improve readability and surface intent in the generated ABI. Consider declaring it as `mapping(address vault => bool approved)`.

Internal helpers reachable as `internal` instead of `private`. The helpers `_collectFees`, `_depositToVault`, `_withdrawFromVault`, and `_cctpBurn` in both `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` are declared `internal` but are only called from within the same contract. Neither contract is intended to be inherited from. Consider tightening the visibility to `private` to reflect actual usage and prevent accidental exposure if an inheritance relationship is added later.

Update: Resolved at commit [2b658e5](#). The team stated:

Resolved all four sub-items:

- **Floating pragma:** Pinned both contracts to `pragma solidity 0.8.28;` to match `foundry.toml`.

- **Non-explicit imports:** Switched all imports to named form (e.g. `import {Ownable2Step} from "..."`).
- **Unnamed mapping parameters:** `approvedVaults` is now `mapping(address vault => bool approved)` and `burnOwner` is now `mapping(bytes32 nonce => address owner)`.
- **Internal helpers:** Tightened `_collectFees`, `_depositToVault`, `_withdrawFromVault`, and `_cctpBurn` from `internal` to `private` in both contracts. `_authorizeUpgrade` remains `internal` as required by the `UUPSUpgradeable` override.

N-04 Missing Documentation On Local External Interfaces

The locally declared CCTP and Aave interface functions in `QuicknodeEarn.sol` and `QuicknodeEarnProxy.sol` (`IPool.supply`, `IPool.withdraw`, `IMessageTransmitter.receiveMessage`, `ITokenMessengerV2.depositForBurn`) carry no NatSpec. The rest of the codebase is well commented, so these stand out as the only places where reviewers and integrators have to consult upstream Aave V3 and Circle CCTP v2 documentation to understand the expected semantics and parameter contracts.

Consider adding NatSpec to the locally declared interface functions, capturing the expected upstream behavior and any version-specific assumptions. In particular for `depositForBurn`, where the contract intentionally tolerates ABI variation between CCTP v2 proxy versions, a short note on the rationale for the low-level call would help future readers.

Update: Resolved at commit [26c3415](#). The team stated:

Resolved. Added NatSpec to all functions in the two locally declared CCTP interfaces `IMessageTransmitter.receiveMessage`, `ITokenMessengerV2.depositForBurnWithHook`, capturing parameter semantics, return values, and any version-specific assumptions.

Per the audit's specific call-out, the `ITokenMessengerV2` block also documents why `_cctpBurn` uses a low-level `call` instead of a typed interface call (some CCTP V2 proxy deployments return additional data that would cause a typed call's ABI decoder to revert).

Aave interfaces `IPool.supply`, `IPool.withdraw` were not touched because the Aave integration was removed from the contracts entirely (see the post-Aave-removal commit on this branch).

Conclusion

The audited codebase implements a non-custodial yield optimizer for USDC that routes user capital across whitelisted Morpho ERC-4626 vaults and Aave V3, with cross-chain movement powered by Circle's CCTP v2.

The recommendations in this report center on tightening on-chain validation around the documented trust assumptions, so that the contract enforces the boundaries that the project's off-chain roles and documentation intend. A second theme covers general code quality and consistency improvements. Together, these changes are intended to reduce the surface that depends on off-chain correctness alone.

We would like to thank the Quicknode team for their responsiveness, clear communication, and continued support throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.